🐧 Линуксийн цөмийн модуль (Linux Kernel Modules)

Энэ төслийн хүрээнд та Линуксийн цөмийн модуль (kernel module) хэрхэн үүсгэж, Линуксийн цөмд ачаалж (load) оруулахыг сурах болно. Энэхүү төслийг энэ номтой хамт өгөгдсөн Линуксийн виртуал машин (Linux virtual machine) ашиглан хийж болно. Та эдгээр С програмуудыг бичихдээ ямар нэгэн текст засварлагч (editor) ашиглаж болох ч, програмуудыг хөрвүүлэх (compile) болон цөмийн модуль удирдах командуудыг терминал (terminal) дээр бичиж гүйцэтгэх шаардлагатай.

Та удахгүй ойлгох болно: цөмийн модуль хөгжүүлэхийн давуу тал нь цөмтэй шууд харилцан ажиллах (kernel functions-ийг шууд дуудах) боломжийг олгодог бөгөөд энэ нь харьцангуй хялбар арга юм. Гэхдээ та цөмийн код бичиж байгаа гэдгээ үргэлж санах хэрэгтэй. Энэ нь кодонд гарсан аливаа алдаа системийг унтраах (crash) аюултай гэсэн үг. Аз болоход та виртуал машин ашиглаж байгаа тул, хамгийн муу тохиолдолд системийг дахин ачаалах (reboot) л шаардлагатай болно.

🔹 I хэсэг – Цөмийн модуль үүсгэх (Creating Kernel Modules)

Энэ төслийн эхний хэсэгт та Линуксийн цөмд модуль үүсгэж оруулах дараалсан алхмуудыг дагана.

Одоогоор ачаалагдсан бүх цөмийн модулиудын жагсаалтыг харахын тулд дараах командыг ашиглана:

```
lsmod
```

Энэ команд нь ачаалагдсан бүх цөмийн модулиудыг 3 баганад харуулна:

- Нэр (name)
- Хэмжээ (size)
- Аль модульд ашиглагдаж буй эсэх (where the module is being used)

Дараах `simple.c` нэртэй програм нь цөмийн маш энгийн модулийн жишээ бөгөөд модуль ачаалагдах болон устах үед тохирох мессеж хэвлэнэ. (Энэ файл нь номын эх кодын хамт ирсэн.)

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>

/* This function is called when the module is loaded. */
int simple_init(void) {
    printk(KERN_INFO "Loading Module\n");
    return 0;
```

```
}

/* This function is called when the module is removed. */
void simple_exit(void) {
    printk(KERN_INFO "Removing Module\n");
}

/* Macros for registering module entry and exit points. */
module_init(simple_init);
module_exit(simple_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple Module");
MODULE_AUTHOR("SGG");
```

`simple_init()` функц нь модулийн оролтын цэг (module entry point) бөгөөд модуль цөмд ачаалагдах үед дуудагдана.
`simple_exit()` функц нь модулийн гарах цэг (module exit point) бөгөөд модуль цөмөөс хасагдах үед дуудагдана.

Модулийн оролтын функц нь `int` төрөлтэй утга буцаана:

- `0` – амжилттай
- Бусад утга – алдаа

Модулийн гарах функц нь `void` төрөлтэй бөгөөд параметр хүлээн авахгүй.

Дараах хоёр макро нь эдгээр функцуудыг цөмд бүртгэдэг:

```
module_init()
module_exit()
```

Мөн анзаарвал, эдгээр функцууд `printk()` функцийг ашиглаж байна.
`printk()` нь цөмийн түвшний `printf()` гэж ойлгож болно. Гэхдээ хэвлэсэн мэдээлэл нь дэлгэц дээр биш, харин цөмийн лог буфер (kernel log buffer) руу ордог. Үүнийг `dmesg` командаар харж болно.

`printk()`-ийн онцлог нь мессежийн ач холбогдлын түвшинг зааж өгч болдог. Энд `KERN_INFO` ашигласан бөгөөд энэ нь мэдээллийн түвшний (informational) мессеж гэсэн үг юм.

Сүүлчийн мөрүүд:

```
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple Module");
MODULE_AUTHOR("SGG");
```

Эдгээр нь:

- Програмын лиценз
- Модулийн тайлбар
- Зохиогчийн нэр

гэсэн мэдээллийг агуулна. Манай төсөлд зайлшгүй хэрэгтэй биш ч, стандарт практик тул оруулж байна.

🔹 Модуль компайл хийх

Энэ модуль нь тухайн төсөлд дагалдаж ирсэн `Makefile` ашиглан компайл хийгдэнэ. Компайл хийхийн тулд:

```
make
```

Энэ командын үр дүнд хэд хэдэн файл үүснэ.
Тэдгээрийн дотроос **simple.ko** файл нь компайл хийгдсэн цөмийн модуль юм.

🔹 Цөмийн модуль ачаалах ба устгах (Loading and Removing Kernel Modules)

Модулийг цөмд ачаалахын тулд:

```
sudo insmod simple.ko
```

Ачаалагдсан эсэхийг шалгахын тулд:

```
lsmod
```

`simple` гэсэн нэртэй модуль гарч ирэх ёстой.

Модуль ачаалагдах үед модулийн оролтын функц дуудагдана. Логийг харахын тулд:

```
dmesg
```

Та **"Loading Module"** гэсэн мессежийг харах ёстой.

Модулийг устгахын тулд:

```
sudo rmmod simple
```

(.ko өргөтгөлийг бичих шаардлагагүй)

Дараа нь дахин `dmesg` ажиллуулж, **"Removing Module"** гарч ирж байгаа эсэхийг шалгана.

Цөмийн лог буфер хурдан дүүрдэг тул үе үе цэвэрлэх нь зүйтэй:

```
sudo dmesg -c
```

✅ I хэсгийн даалгавар

Дээр тайлбарласан алхмуудыг дагаж:

- Цөмийн модуль үүсгэх
- Модуль ачаалах
- Модуль устгах

бүх үйлдлийг хийж үз.
`dmesg` ашиглан логийг шалгаж, зөв ажиллаж байгааг баталгаажуул.

🔹 II хэсэг – Цөмийн өгөгдлийн бүтэц (Kernel Data Structures)

Энэ хэсэгт бид цөмийн модулийг өөрчилж, Линуксийн **холбоос жагсаалт (linked list)** өгөгдлийн бүтцийг ашиглах болно.

Линуксийн цөм нь хэд хэдэн өгөгдлийн бүтэц санал болгодог. Энд бид **дугуй, хоёр талт холбоост жагсаалт (circular doubly linked list)** ашиглана.

Энэ бүхэн `<linux/list.h>` файлд тодорхойлогдсон байдаг тул та түүнийг нээж үзэхийг зөвлөе.

🔹 Холбоост жагсаалтад ашиглах struct тодорхойлох

Дараах struct нь төрсөн өдрийг илэрхийлнэ:

```
struct birthday {
    int day;
    int month;
    int year;
    struct list_head list;
};
```

`struct list_head list;` гэдэг гишүүн нь тухайн элементүүдийг холбоост жагсаалтад оруулах зориулалттай.

`list_head` бүтэц нь `<linux/types.h>` дотор тодорхойлогдсон бөгөөд:

- `next`
- `prev`

гэсэн хоёр заагч (pointer)-тай.

Энэ аргаар struct дотор жагсаалтыг “шигтгэж” өгснөөр Линуксийн макро функцуудыг ашиглан өгөгдлийг хялбар удирдах боломж бүрддэг.

Холбоост жагсаалтад элемент нэмэх

Жагсаалтын толгойг дараах макроор тодорхойлно:

```
static LIST_HEAD(birthday_list);
```

Энэ нь `birthday_list` нэртэй, `struct list_head` төрөлтэй хувьсагчийг тодорхойлж, анхны утгаар нь инициализ хийж өгнө.

Дараа нь шинэ элемент үүсгэнэ:

```
struct birthday *person;
person = kmalloc(sizeof(*person), GFP_KERNEL);
person->day = 2;
person->month = 8;
person->year = 1995;
INIT_LIST_HEAD(&person->list);
```

`kmalloc()` нь хэрэглэгчийн түвшний `malloc()`-ын цөмийн хувилбар юм.
`GFP_KERNEL` нь ердийн цөмийн санах ойгоос санах ой нөөцөлж байгааг илэрхийлнэ.

Дараа нь жагсаалтын төгсгөлд нэмнэ:

```
list_add_tail(&person->list, &birthday_list);
```

Холбоост жагсаалтыг гүйцэж харах (Traversal)

Жагсаалтыг тойрч харахын тулд:

```
struct birthday *ptr;
list_for_each_entry(ptr, &birthday_list, list) {
    /* энд ptr нь дараагийн birthday struct руу заана */
}
```

Холбоост жагсаалтаас элемент устгах

Элементийг устгах:

```
list_del(struct list_head *element);
```

Жагсаалтыг бүхэлд нь устгах хамгийн хялбар арга:

```
struct birthday *ptr, *next;
```

```
list_for_each_entry_safe(ptr, next, &birthday_list, list) {
    list_del(&ptr->list);
    kfree(ptr);
}
```

`kfree()` нь `kmalloc()`-аар авсан санах ойг буцааж чөлөөлнө.

⚠️ Цөмийн код дээр санах ойн менежмент маш чухал.
Санах ойг чөлөөлөхгүй орхивол memory leak үүснэ.

✅ II хэсгийн даалгавар

- Модулийн `init` функц дотор 5 ширхэг `struct birthday` үүсгэж холбоост жагсаалт болго.
- Жагсаалтыг тойрч гүйж, бүх утгыг `printk()` ашиглан лог руу хэвлэ.
- `dmesg` ашиглан зөв хэвлэгдсэн эсэхийг шалга.
- Модулийн `exit` функц дотор бүх элементүүдийг устгаж, `kfree()`-ээр санах ойг чөлөөл.
- Дахин `dmesg` ашиглан жагсаалт амжилттай устсан эсэхийг шалга.

1) `birthday_list.c` (Бүрэн модуль)

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/slab.h>      // kmalloc, kfree
#include <linux/list.h>      // linked list macros

struct birthday {
    int day;
    int month;
    int year;
    struct list_head list;
};

static LIST_HEAD(birthday_list);

static int __init birthday_init(void)
{
    struct birthday *person;
    int i;

    printk(KERN_INFO "Birthday Module: Loading...\n");

    /* 5 ширхэг элемент үүсгэж жагсаалтанд нэмнэ */
    for (i = 0; i < 5; i++) {
        person = kmalloc(sizeof(*person), GFP_KERNEL);
        if (!person) {
            printk(KERN_ERR "Birthday Module: kmalloc failed!\n");
            return -ENOMEM;
```

```
        }

        /* Жишээ өгөгдөл */
        person->day = (i + 1) * 2;          // 2,4,6,8,10
        person->month = 8 + i;              // 8,9,10,11,12
        person->year = 1995 + i;            // 1995..1999

        INIT_LIST_HEAD(&person->list);
        list_add_tail(&person->list, &birthday_list);
    }

    /* Жагсаалтыг гүйцэж хэвлэх */
    {
        struct birthday *ptr;
        printk(KERN_INFO "Birthday Module: Listing birthdays:\n");
        list_for_each_entry(ptr, &birthday_list, list) {
            printk(KERN_INFO "  - %02d/%02d/%04d\n", ptr->day, ptr->month,
ptr->year);
        }
    }

    return 0;
}

static void __exit birthday_exit(void)
{
    struct birthday *ptr, *next;

    printk(KERN_INFO "Birthday Module: Removing...\n");

    /* Бүх элементийг safe байдлаар устгаж санах ойг чөлөөлнө */
    list_for_each_entry_safe(ptr, next, &birthday_list, list) {
        printk(KERN_INFO "  deleting: %02d/%02d/%04d\n", ptr->day, ptr-
>month, ptr->year);
        list_del(&ptr->list);
        kfree(ptr);
    }

    printk(KERN_INFO "Birthday Module: Removed. List cleared.\n");
}

module_init(birthday_init);
module_exit(birthday_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Kernel Linked List Example (Birthdays)");
MODULE_AUTHOR("Batbayar");
```

2) **Makefile**

Тухайн `birthday_list.c` файлтайгаа **нэг хавтасанд** дараах Makefile-ийг хийнэ:

```
obj-m += birthday_list.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
```

```
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

3) Компайл хийх, ачаалах, шалгах командууд

Тухайн хавтас руу орж:

```
make
```

Ачаалахын өмнө лог цэвэрлэх (сонголтоор):

```
sudo dmesg -c
```

Модуль ачаалах:

```
sudo insmod birthday_list.ko
```

Ачаалагдсан эсэхийг шалгах:

```
lsmod | grep birthday_list
```

Лог харах:

```
dmesg
```

Та ойролцоогоор ийм мөрүүдийг харна:

- `Birthday Module: Loading...`
- `Birthday Module: Listing birthdays:`
- `- 02/08/1995` гэх мэт 5 мөр

Модуль устгах:

```
sudo rmmod birthday_list
```

Дахиад лог харах:

```
dmesg
```

Устгах үеийн `deleting:` гэсэн 5 мөр гарсан байх ёстой.

4) Хамгийн түгээмэл алдаа (хурдан оноош)

- **`insmod: ERROR: could not insert module ... Operation not permitted`**
  - `sudo`-той ажиллуулж байгаа эсэхээ шалга.
- **`Invalid module format`**
  - Kernel-ийн хувилбар, header/build тохирохгүй байж магадгүй (`uname -r` vs headers).

- **VM дээр Secure Boot / module signing шаардлага**
  - Зарим distro дээр асуудал өгч болно.

жишээ kernel module өгье. Ингэснээр:

1. **2000 оноос өмнөх төрсөн огноог тусгайлан тэмдэглэнэ**
2. **procfs-ээр дамжуулж хэрэглэгчийн талд жагсаалтыг уншина** (`/proc/birthdays`)
3. **`list_add()` (толгой дээр) vs `list_add_tail()` (сүүлд) ялгааг харьцуулна**

✅ Сайжруулсан модуль: `birthday_proc.c`

```
#include <linux/init.h>
    printk(KERN_INFO "Birthday Proc Module: Loading...\n");

    /* Толгой дээр нэмэх (list_add) */
    p = kmalloc(sizeof(*p), GFP_KERNEL);
    p->day = 1; p->month = 1; p->year = 1995; // pre-2000
    INIT_LIST_HEAD(&p->list);
    list_add(&p->list, &birthday_list);

    p = kmalloc(sizeof(*p), GFP_KERNEL);
    p->day = 12; p->month = 5; p->year = 1998; // pre-2000
    INIT_LIST_HEAD(&p->list);
    list_add(&p->list, &birthday_list);

    /* Сүүлд нэмэх (list_add_tail) */
    p = kmalloc(sizeof(*p), GFP_KERNEL);
    p->day = 3; p->month = 7; p->year = 2001;
    INIT_LIST_HEAD(&p->list);
    list_add_tail(&p->list, &birthday_list);

    p = kmalloc(sizeof(*p), GFP_KERNEL);
    p->day = 20; p->month = 9; p->year = 2005;
    INIT_LIST_HEAD(&p->list);
    list_add_tail(&p->list, &birthday_list);

    /* procfs үүсгэх: /proc/birthdays */
    proc_entry = proc_create(PROC_NAME, 0, NULL, &birthdays_fops);
    if (!proc_entry) {
        printk(KERN_ERR "Failed to create /proc/%s\n", PROC_NAME);
        return -ENOMEM;
    }

    printk(KERN_INFO "Birthday Proc Module: /proc/%s created\n",
PROC_NAME);
    return 0;
}

static void __exit birthday_exit(void)
{
```

```
    struct birthday *ptr, *next;

    printk(KERN_INFO "Birthday Proc Module: Removing...\n");

    /* procfs устгах */
    if (proc_entry)
        proc_remove(proc_entry);

    /* Жагсаалтыг цэвэрлэх */
    list_for_each_entry_safe(ptr, next, &birthday_list, list) {
        printk(KERN_INFO " deleting: %02d/%02d/%04d\n",
               ptr->day, ptr->month, ptr->year);
        list_del(&ptr->list);
        kfree(ptr);
    }

    printk(KERN_INFO "Birthday Proc Module: Removed.\n");
}

module_init(birthday_init);
module_exit(birthday_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Batbayar");
MODULE_DESCRIPTION("Birthday list with procfs + list_add vs list_add_tail
demo");
```

🧪 Яаж турших вэ?

1 Компайл

```
make
```

2 Ачаалах

```
sudo insmod birthday_proc.ko
```

3 Хэрэглэгчийн талаас унших (procfs)

```
cat /proc/birthdays
```

Жишээ гаралт:

```
== Birthday List ==
  * 12/05/1998  (pre-2000)
  * 01/01/1995  (pre-2000)
  - 03/07/2001
  - 20/09/2005
```

Энд:

- `list_add()`-аар нэмсэн хоёр элемент **толгой дээр** орж ирсэн (дараалал урвуулагдана)
- `list_add_tail()`-аар нэмсэн нь **сүүлд** дарааллаараа нэмэгдсэн

4 Устгах

```
sudo rmmod birthday_proc
```

🧠 Гол ойлголтууд (товч дүгнэлт)

◇ `list_add()` vs `list_add_tail()`

Функц	Байрлал	Ашиглах үед
`list_add()`	Толгой дээр	LIFO (stack) маягаар нэмэх
`list_add_tail()`	Сүүлд	FIFO (queue) маягаар нэмэх

◇ procfs яагаад хэрэгтэй вэ?

- Цөмийн доторх өгөгдлийг **хэрэглэгчийн түвшинд унших** энгийн арга
- Debug хийх, monitoring хийхэд маш тохиромжтой
- `/proc/...` файл руу `cat` хийхэд шууд уншигдана